

ООП 1

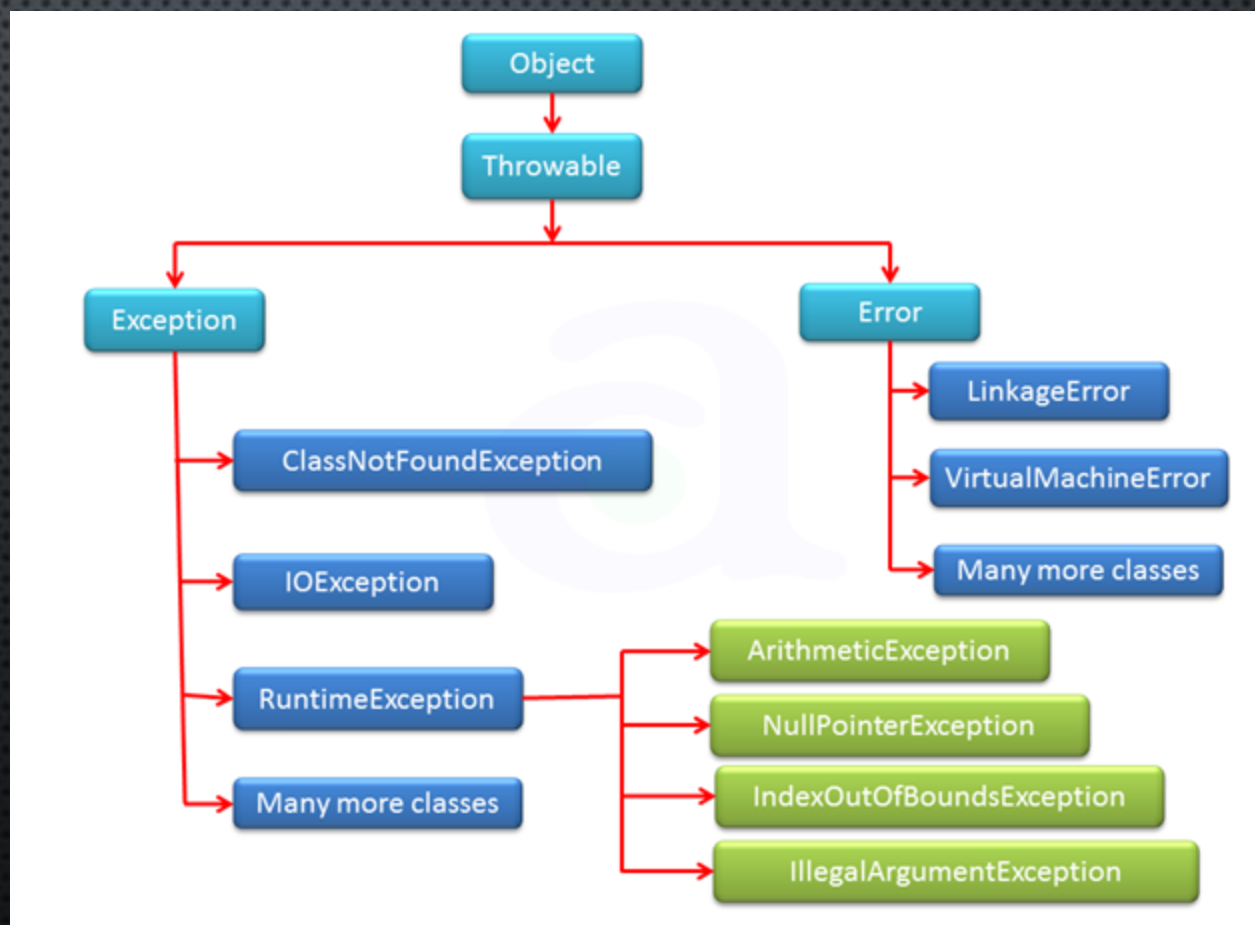
ОБРАБОТКА НА ИЗКЛЮЧЕНИЯ.

СЪЩНОСТ

- ОБРАБОТКА НА ГРЕШКИ — АНОМАЛНИ СЪБИТИЯ
- РЕАКЦИЯ ПРИ НАСТЪПИЛА ГРЕШКА В RUN-TIME

ЙЕРАРХИЯ НА КЛАСОВЕ ЗА ГРЕШКИ

- ТИПОВЕ ГРЕШКИ
 - ПРОВЕРЕНИ
 - НЕПРОВЕРЕНИ (RUNTIMEEXCEPTION)
 - ГРЕШКИ



ОБРАБОТКИ НА ИЗКЛЮЧЕНИЯ

- БЛОК — TRY-CATCH-FINALLY

```
TRY {  
    ....  
} CATCH (FileNotFoundException e) {  
    ....  
}
```

С МНОЖЕСТВО CATCH БЛОКОВЕ

```
TRY {  
    .....  
} CATCH (EXCEPTIONEXAMPLE1 E1) {  
    .....  
} CATCH (EXCEPTIONEXAMPLE2 E2) {  
    .....
```

С МНОЖЕСТВО CATCH БЛОКОВЕ

- `CATCH (IOException|SQLException EX)`

ПРИОРИТЕТ НА CATCH КЛАУЗИТЕ

- ЙЕРАРХИЯ НА КЛАСОВЕТЕ !!!

ВЪТРЕШНИ TRY БЛОКОВЕ

- ПРИ НЕОБХОДИМОСТ ОТ РАЗКЛОНЯВАНЕ НА ЛОГИКАТА ЗА ОБРАБОТКА

ВЪТРЕШНИ TRY БЛОКОВЕ

```
PUBLIC CLASS NESTEDTRYEXAMPLE {  
    PUBLIC STATIC VOID MAIN(STRING[] ARGS) {  
        TRY {  
            // ВЪНШЕН TRY БЛОК  
            SYSTEM.OUT.PRINTLN("STARTING OUTER TRY BLOCK...");  
  
            TRY {  
                // ВЪТРЕШЕН TRY БЛОК  
                INT RESULT = 10 / 0; // ТОВА ЩЕ ХВЪРЛИ ARITHMETICEXCEPTION  
                SYSTEM.OUT.PRINTLN("RESULT: " + RESULT);  
            } CATCH (ARITHMETICEXCEPTION E) {  
                SYSTEM.OUT.PRINTLN("CAUGHT AN ARITHMETICEXCEPTION IN THE INNER TRY BLOCK:  
" + E.GETMESSAGE());  
            }  
        }  
    }  
}
```

```
        STRING STR = NULL;  
        SYSTEM.OUT.PRINTLN(STR.LENGTH()); // ТОВА ЩЕ ХВЪРЛИ NULLPOINTEREXCEPTION  
  
    } CATCH (NULLPOINTEREXCEPTION E) {  
        SYSTEM.OUT.PRINTLN("CAUGHT A NULLPOINTEREXCEPTION IN THE OUTER TRY BLOCK: "  
+ E.GETMESSAGE());  
    } FINALLY {  
        SYSTEM.OUT.PRINTLN("OUTER FINALLY BLOCK EXECUTED.");  
    }  
  
    SYSTEM.OUT.PRINTLN("PROGRAM CONTINUES...");  
}
```

FINALLY БЛОК

- ЗАДЪЛЖИТЕЛНОСТ НА ИЗПЪЛНЕНИЕ

```
TRY{  
.....  
} CATCH .... {  
.....  
} FINALLY {  
.....  
}
```

КЛЮЧОВАТА ДУМА THROW

- ЕКСПЛИЦИТНО ГЕНЕРИРАНЕ НА ГРЕШКА
 - **throw** new ArithmeticException(....
 - ПРОВЕРЕНИ
 - НЕПРОВЕРЕНИ
 - СОБСТВЕНИ

ПРЕДАВАНЕ НА ГРЕШКИ

- ПРЕДАВАНЕ КЪМ ВИКАЩИЯТ МЕТОД
 - НАГОРЕ ПО ЙЕРАРХИЯТА

КЛЮЧОВАТА ДУМА THROWS

- ДЕКЛАРАЦИЯ НА ВЪЗМОЖНА ГРЕШКА
 - `VOID READFROMNETWORK() THROWS IOEXCEPTION {`

ДЕФИНИРАНЕ НА ИЗКЛЮЧЕНИЯ ПРИ НАСЛЕДЯВАНЕ НА МЕТОД

- АКО МЕТОДЪТ НА СУПЕРКЛАСА НЕ ДЕKLАРИРА ИЗКЛЮЧЕНИЕ
 - НЕ МОЖЕ ДА ДЕKLАРИРА ПРОВЕРЕНОТО ИЗКЛЮЧЕНИЕ, НО МОЖЕ ДА ДЕKLАРИРА НЕПРОВЕРЕНО ИЗКЛЮЧЕНИЕ.
- АКО МЕТОДЪТ НА СУПЕРКЛАСА ДЕKLАРИРА ИЗКЛЮЧЕНИЕ
 - МОЖЕ ДА ДЕKLАРИРА СЪЩОТО ИЗКЛЮЧЕНИЕ НЕГОВ ПОДКЛАС ИЛИ БЕЗ ИЗКЛЮЧЕНИЕ, НО НЕ МОЖЕ ДА ДЕKLАРИРА ИЗКЛЮЧЕНИЕ ОТ ПО ВИСОКО НИВО.

СОБСТВЕНИ (USER-DEFINED) ИСКЛЮЧЕНИЯ

- СОБСТВЕНА ЛОГИКА ЗА АНОМАЛИЯ

СОБСТВЕННИ (USER-DEFINED) ИСКЛЮЧЕНИЯ

```
PUBLIC CLASS INVALIDUSERINPUTEXCEPTION EXTENDS EXCEPTION {  
    PUBLIC INVALIDUSERINPUTEXCEPTION(STRING MESSAGE) {  
        SUPER(MESSAGE);  
    }  
  
    PUBLIC INVALIDUSERINPUTEXCEPTION(STRING MESSAGE, THROWABLE CAUSE) {  
        SUPER(MESSAGE, CAUSE);  
    }  
}
```

СОБСТВЕННИ (USER-DEFINED) ИЗКЛЮЧЕНИЯ

```
PUBLIC CLASS USERINPUTVALIDATOR {  
    PUBLIC STATIC VOID MAIN(String[] args) {  
        TRY {  
            VALIDATEUSERNAME("INV@LIDUSERNAME!");  
        } CATCH (INVALIDUSERINPUTEXCEPTION E) {  
            SYSTEM.OUT.PRINTLN("CAUGHT AN EXCEPTION: " + E.GETMESSAGE());  
        }  
    }  
}
```

```
PUBLIC STATIC VOID VALIDATEUSERNAME(String username) THROWS  
INVALIDUSERINPUTEXCEPTION {  
    IF (!username.MATCHES("^[a-zA-Z0-9_]+$")) {  
        THROW NEW INVALIDUSERINPUTEXCEPTION("INVALID USERNAME: ONLY  
ALPHANUMERIC CHARACTERS AND UNDERSCORES ARE ALLOWED.");  
    }  
    SYSTEM.OUT.PRINTLN("USERNAME IS VALID.");  
}
```